Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Build Your Own Small OS (xv6)

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. Building and Booting xv6 **2**

3 Part 2. Use xv6 as the Unix It Is **2**

3.1 Observe answers 2

4 Part 3. Reading the Source **3**

4.1 Observe answers 3

5 Conclusion **3**

1. Purpose of the Work

The purpose of this laboratory work is to build and boot xv6, use it as a small Unix-like operating system, and connect observable shell behavior with the source code in the `user/` and `kernel/` directories.

2. Part 1. Building and Booting xv6

The xv6 source tree was built with the RISC-V toolchain and started in QEMU. The actual boot reached the shell successfully.

```
xv6 kernel is booting

init: starting sh
$
```

Figure 1.1 – Actual xv6 boot excerpt showing that the shell started successfully

The build compiled kernel and user programs, created the file-system image with `mkfs`, and linked the kernel. A linker warning about an RWX segment was observed, but the kernel booted and the shell was usable.

3. Part 2. Use xv6 as the Unix It Is

The xv6 image contains ordinary user programs such as `ls`, `cat`, and `echo`. Basic command execution and pipes behave similarly to Unix/Linux even though the shell is much smaller.

3.1. Observe answers

1. Three programs included with xv6 are `ls`, `cat`, and `echo`.
2. For a pipeline such as `ls | grep c` to work, the OS needs processes and inter-process communication. The shell starts separate processes and connects the output of one process to the input of another through a pipe.
3. The xv6 shell is much simpler than Bash, but basic Unix mechanisms such as program execution, standard input/output, and pipes work according to the same core model.

4. Part 3. Reading the Source

The source of `user/cat.c` and the kernel system-call implementation were inspected.

```
sys_read  -> kernel/sysfile.c, line 69
sys_write -> kernel/sysfile.c, line 83
```

Figure 3.1 – Locations of the read and write system-call handlers found in the xv6 source

4.1. Observe answers

1. `user/cat.c` uses `read()` to obtain bytes from a file descriptor, `write()` to send bytes to a file descriptor, `open()` to open a file, `close()` to release a descriptor, and `exit()` to terminate the process.
2. `sys_read` is implemented in `kernel/sysfile.c` at line 69, and `sys_write` starts at line 83 in the inspected tree.
3. Code under `kernel/` runs as privileged operating-system code and manages resources. Code under `user/` consists of ordinary programs that request kernel services through system calls.

5. Conclusion

This work confirmed that xv6 is a small but real Unix-like operating system. Building and booting it exposed the complete path from source code to a running shell, while the source inspection connected user-level calls such as `read()` and `write()` with privileged kernel implementations.